

MÓDULO

BASE DE DADOS MYSQL

UNIDADE

ELIMINAÇÃO DE DADOS EM MYSQL

ÍNDICE

OBJETIVOS.....	3
INTRODUÇÃO.....	4
1. FAZER UMA CÓPIA DE SEGURANÇA	5
2. APAGAR LINHAS	8
3. APAGAR TABELAS.....	18
4. APAGAR BASES DE DADOS	22
CONCLUSÃO	25
AUTOAVALIAÇÃO	27
SOLUÇÕES.....	31
PROPOSTAS DE DESENVOLVIMENTO DO ESTUDO.....	32
BIBLIOGRAFIA	33

OBJETIVOS

Com esta unidade didática, pretende-se que desenvolva os seguintes objetivos de aprendizagem:

- Aprender a fazer uma cópia de segurança de toda a base de dados.
- Saber como excluir linhas ou registos específicos.
- Saber como excluir tabelas inteiras.
- Saber como excluir bases de dados inteiras.

INTRODUÇÃO

Da mesma forma que limpa a sua casa, deverá fazer o mesmo com os ficheiros fotográficos, discos rígidos, bases de dados, etc. Não se trata de eliminar tudo, apenas de investir o tempo necessário para procurar e decidir o que ainda lhe é necessário e o que não é.

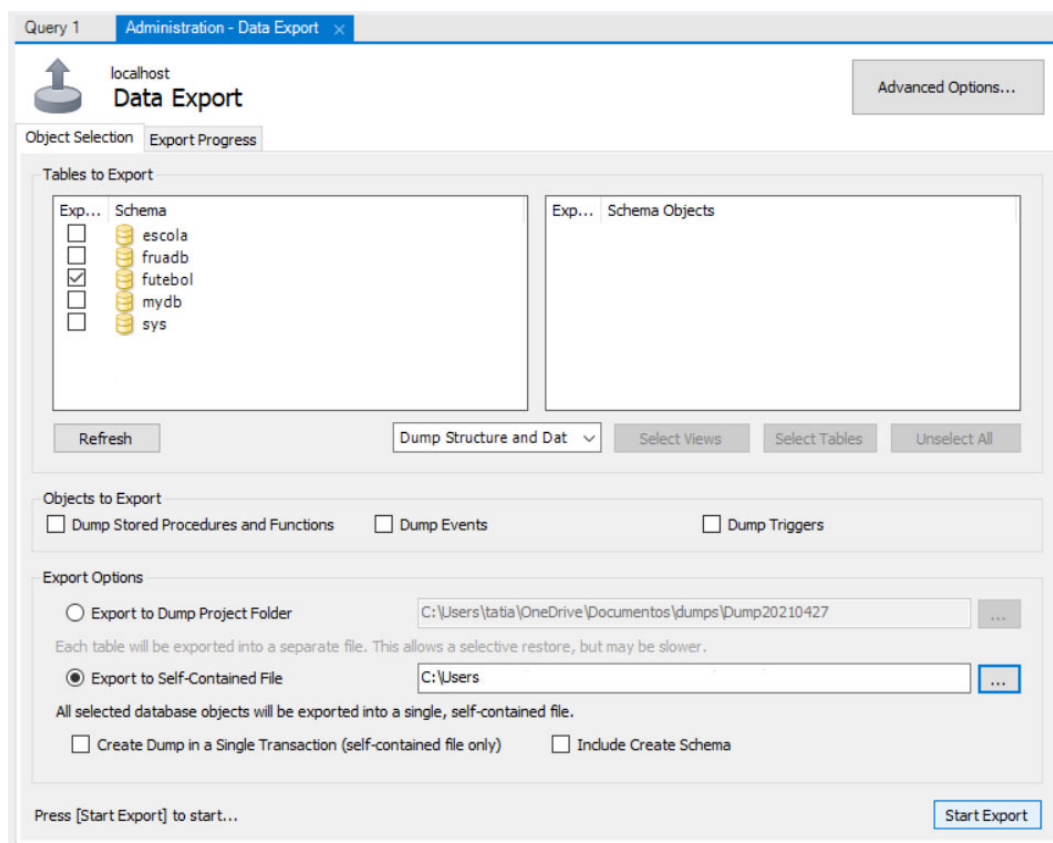
Assim, nesta unidade didática, aprenderá a fazer essa mesma pesquisa e a eliminar dados das bases de dados, sempre com a maior atenção e prudência.

1. FAZER UMA CÓPIA DE SEGURANÇA

Como nesta unidade irá eliminar dados, tabelas e a base de dados, algo nada recomendável, a não ser que tenha a certeza absoluta do que faz, o melhor é fazer uma cópia de segurança (um backup) da base de dados antes de começar. Desta forma, poderá trabalhar nesta unidade sem receios e com todas as garantias.

A primeira etapa para realizar o backup será abrir o MySQL Workbench e clicar duas vezes na conexão para abrir a base de dados do futebol. Em seguida, é possível que tenha também de introduzir uma password (consoante as definições).

Depois, irá aparecer a janela padrão, onde poderia escrever normalmente novas consultas, mas deverá selecionar no separador "Server" a opção "Data Export", que irá permitir exportar os dados das bases de dados. Nesta janela aparecerá a lista de bases de dados disponíveis no servidor. Neste caso, selecione a base de dados que tem vindo a utilizar ao longo das unidades, chamada "futebol". A seguir, entre todas as opções marque a opção "Export to Self-Contained File" e escolha o caminho onde pretende que o ficheiro seja guardado.



Depois de realizar estas etapas, clique em “StartExport”. Em seguida, será solicitada novamente a password e o backup será iniciado. Poderá confirmar se o processo foi concluído através da barra de progresso – quando o processo estiver concluído, a barra de progresso estará completamente verde e aparecerá uma mensagem a avisar que a exportação foi concluída.

No final desta exportação, é criado um ficheiro de texto com as instruções SQL, que permite a reconstrução de toda a base de dados. Para verificar se o backup acabado de fazer não apresenta problemas, pode procurar o ficheiro criado no caminho que indicou anteriormente ao MySQL Workbench para o criar. A seguir, deverá abri-lo com um editor de texto simples. Como poderá ver, foram criadas as instruções SQL necessárias para a construção das tabelas e para inserir os respetivos dados nas mesmas.


```
82
83 DROP TABLE IF EXISTS `equipa`;
84 /*!40101 SET @saved_cs_client      = @@character_set_client */;
85 /*!50503 SET character_set_client = utf8mb4 */;
86 CREATE TABLE `equipa` (
87   `equipaID` int(11) NOT NULL AUTO_INCREMENT,
88   `nomeEquipa` varchar(45) NOT NULL,
89   `paisID` char(3) NOT NULL,
90   `dataFundacao` date DEFAULT NULL,
91   `ultimoPresidente` varchar(100) DEFAULT NULL,
92   PRIMARY KEY (`equipaID`),
93   KEY `fk_equipa_pais1_idx` (`paisID`),
94   CONSTRAINT `fk_equipa_pais1` FOREIGN KEY (`paisID`) REFERENCES `pais` (`paisID`) ON DELETE NO ACTION ON UPDATE NO ACTION
95 ) ENGINE=InnoDB AUTO_INCREMENT=79 DEFAULT CHARSET=utf8;
96 /*!40101 SET character_set_client = @saved_cs_client */;
97
98 --
99 -- Dumping data for table `equipa`
100 --
101
102 LOCK TABLES `equipa` WRITE;
103 /*!40000 ALTER TABLE `equipa` DISABLE KEYS */;
104 INSERT INTO `equipa` VALUES (2,'Farense','POR',NULL,NULL),(3,'Portimonense','POR',NULL,NULL);
105 /*!40000 ALTER TABLE `equipa` ENABLE KEYS */;
106 UNLOCK TABLES;
107
108 --
109 -- Table structure for table `pais`
110 --
111
112 DROP TABLE IF EXISTS `pais`;
113 /*!40101 SET @saved_cs_client      = @@character_set_client */;
114 /*!50503 SET character_set_client = utf8mb4 */;
115 CREATE TABLE `pais` (
116   `paisID` char(3) NOT NULL,
117   `paisNome` varchar(50) NOT NULL,
```

2. APAGAR LINHAS

Assim que a cópia de segurança estiver realizada (guarde-a bem porque irá usá-la novamente mais tarde), já poderá trabalhar com mais confiança na base de dados e eliminar o que for necessário sem medo de perder essa informação para sempre.

Para eliminar registos da base de dados é utilizado o comando **DELETE**. Para saber mais sobre a sua utilização deverá executar a ajuda da linha de comandos, através da instrução **\help DELETE**, que irá exibir a sintaxe a ser utilizada. Mais informações podem ser encontradas na página web oficial: <http://dev.mysql.com/doc/refman/5.1/en/delete.html>.

```
mysql> help DELETE
Name: 'DELETE'
Description:
Syntax:
DELETE is a DML statement that removes rows from a table.

Single-Table Syntax

DELETE [LOW_PRIORITY] [QUICK] [IGNORE] FROM tbl_name
      [PARTITION (partition_name [, partition_name] ...)]
      [WHERE where_condition]
      [ORDER BY ...]
      [LIMIT row_count]

The DELETE statement deletes rows from tbl_name and returns the number
of deleted rows. To check the number of deleted rows, call the
ROW_COUNT() function described in
http://dev.mysql.com/doc/refman/5.7/en/information-functions.html.

Main Clauses

The conditions in the optional WHERE clause identify which rows to
delete. With no WHERE clause, all rows are deleted.

where_condition is an expression that evaluates to true for each row to
be deleted. It is specified as described in
http://dev.mysql.com/doc/refman/5.7/en/select.html.

If the ORDER BY clause is specified, the rows are deleted in the order
that is specified. The LIMIT clause places a limit on the number of
rows that can be deleted. These clauses apply to single-table deletes,
but not multi-table deletes.

Multiple-Table Syntax

DELETE [LOW_PRIORITY] [QUICK] [IGNORE]
      tbl_name[.*] [, tbl_name[.*]] ...
      FROM table_references
      [WHERE where_condition]

DELETE [LOW_PRIORITY] [QUICK] [IGNORE]
      FROM tbl_name[.*] [, tbl_name[.*]] ...
      USING table_references
      [WHERE where_condition]

Privileges
```

```
DELETE [LOW_PRIORITY] [QUICK] [IGNORE]
FROM nomeTabela
[WHERE condição]
[ORDER BY critérios]
[LIMIT numeroLinhas]
```

A palavra **FROM** é obrigatória e, embora o uso da palavra **WHERE** seja opcional, é altamente recomendado, pois permite que apenas os registos que se encaixem numa determinada condição sejam excluídos, evitando assim a eliminação de todos os registos de uma tabela. Geralmente, o **ORDER BY** e o **LIMIT** não são muito utilizados, embora a sua utilização possa ser vantajosa em alguns casos.

Para colocar em prática a sintaxe vista acima, abra o MySQL Workbench e escreva a frase seguinte, com o intuito de apagar “Albânia” da base de dados.

```
DELETE FROM pais WHERE paisID = 'ALB';
```

Caso tudo tenha corrido bem ao executar a sintaxe anterior, tal será indicado pela mensagem **1 row(s) affected**. Verifique se o referido país, de facto, desapareceu com a seguinte consulta de seleção:

```
SELECT * FROM pais WHERE paisID = 'ALB';
```

Tal como esperado, o país desapareceu do banco de dados. Em seguida, tente-se-á fazer o mesmo com “Portugal”.

```
DELETE FROM pais WHERE paisID = 'POR';
```

Como pode observar, “Portugal” foi excluído da base de dados sem problemas. É possível que se questione o que é que aconteceu com as equipas portuguesas, por isso, execute a seguinte consulta para tentar descobrir:

```
SELECT * FROM equipa;
```

Query 1 equipa

Limit to 1000 rows

```

1 • DELETE FROM pais WHERE paisID = 'POR';
2 • SELECT * FROM futebol.equipa;

```

Result Grid

	equipaID	nomeEquipa	paisID	dataFundacao	ultimoPresidente
2		Farense	NULL	NULL	NULL
3		Portimonense	NULL	NULL	NULL
4		Famalicão	NULL	NULL	NULL
5		Gil Vicente	NULL	NULL	NULL
6		Moreirense	NULL	NULL	Vitor Magalhães
7		Sporting de Braga	NULL	1921-01-19	António Salvador
8		Vitória	NULL	1910-11-22	Fernando Oliveira
9		Belenenses SAD	NULL	1919-09-23	Patrick Moraes de Carvalho
10		Benfica	NULL	1904-02-28	Luis Filipe Vieira
11		Sporting	NULL	1906-07-01	Frederico Varandas
12		Marítimo	NULL	NULL	Carlos Pereira
13		Nacional	NULL	NULL	NULL
14		Santa Clara	NULL	NULL	NULL
15		Boavista	NULL	1903-08-01	NULL
16		Paços de Ferreira	NULL	NULL	NULL
17		Porto	NULL	1893-09-28	Pinto da Costa
18		Rio Ave	NULL	NULL	Luis Carlos Andrade
19		Tondela	NULL	NULL	Gilberto Coimbra
20		Arouca	NULL	NULL	NULL

equipa 1 x

Apply Revert

Todas as equipas continuam a existir, mas falta-lhes a indicação do respetivo país. Tal ocorre porque, quando a tabela foi criada, se aplicou a restrição da chave estrangeira **ON DELETE SET NULL**.

```

CONSTRAINT pais_equipa_FK FOREIGN KEY (paisID)
REFERENCES pais (paisID))
ON DELETE SET NULL
ON UPDATE CASCADE

```

Caso a tabela não tivesse sido criada com estas definições e desse erro ao tentar apagar "Portugal" da base de dados, poderia executar-se a query abaixo e depois tentar apagar novamente.

```

ALTER TABLE `futebol`.`equipa`
DROP FOREIGN KEY `fk_equipa_pais1`;
ALTER TABLE `futebol`.`equipa`
CHANGE COLUMN `paisID` `paisID` CHAR(3) NULL ;
ALTER TABLE `futebol`.`equipa`

```

```
ADD CONSTRAINT `fk_equipa_pais1`  
  FOREIGN KEY (`paisID`)  
  REFERENCES `futebol`.`pais` (`paisID`)  
  ON DELETE SET NULL  
  ON UPDATE NO ACTION;
```

A situação teria sido diferente se tivesse indicado que nenhuma ação seria executada com a exclusão. Pode mudar esta situação modificando a restrição de chave estrangeira e voltando a tentar excluir outro país, cujas equipas estejam na tabela das equipas. Ou seja, primeiramente terá de modificar a tabela, alterando a restrição da chave estrangeira, o que será feito com um excerto de código muito semelhante ao anterior; a única diferença será que **ON DELETE** desta vez será acompanhado pelas palavras **NO ACTION**.

```
ALTER TABLE `futebol`.`equipa`  
  DROP FOREIGN KEY `fk_equipa_pais1`;  
ALTER TABLE `futebol`.`equipa`  
  ADD CONSTRAINT `fk_equipa_pais1`  
    FOREIGN KEY (`paisID`)  
    REFERENCES `futebol`.`pais` (`paisID`)  
    ON DELETE NO ACTION  
    ON UPDATE NO ACTION;
```

Em seguida, tente novamente com outro país que tenha equipas na tabela da seleção, como a Itália. Neste caso, não deverá ser possível excluir este país, pois foram alteradas as restrições de chave estrangeira da tabela “equipa”.

```
DELETE FROM pais WHERE paisID = 'ITA';
```

De facto, neste caso, o país não pode ser removido, devido à restrição de chave estrangeira. Então, o MySQL envia a seguinte mensagem de erro:

```
ERROR 1451 (23000): Cannot delete or update a parent row: a
foreignkeyconstraint fails ('futebol'. 'equipa', CONSTRAINT
'pais_equipa_FK' FOREIGN KEY ('paisID') REFERENCES 'pais'
('paisID') ON DELETE NO ACTION ON UPDATE CASCADE)
```

Como referido anteriormente, depois de apagar os dados não há como voltar atrás e os dados perdidos não são recuperáveis. Ser mais ou menos restritivo quando se trata de eliminar dados depende dos parâmetros aplicados desde o momento em que as tabelas são criadas.

Foi também referido antes que, ao utilizar a instrução **DELETE** sem a cláusula **WHERE**, todos os registos de uma tabela são excluídos. Poderá verificá-lo agora, apagando a tabela “campeonato”, simplesmente porque a sua chave primária não é uma chave estrangeira de nenhuma outra tabela, logo, não deverá ter problemas ao apagar os registos da mesma. Em primeiro lugar, deve verificar se esta tabela contém dados com a seguinte consulta:

```
SELECT COUNT(*) FROM campeonato;
```

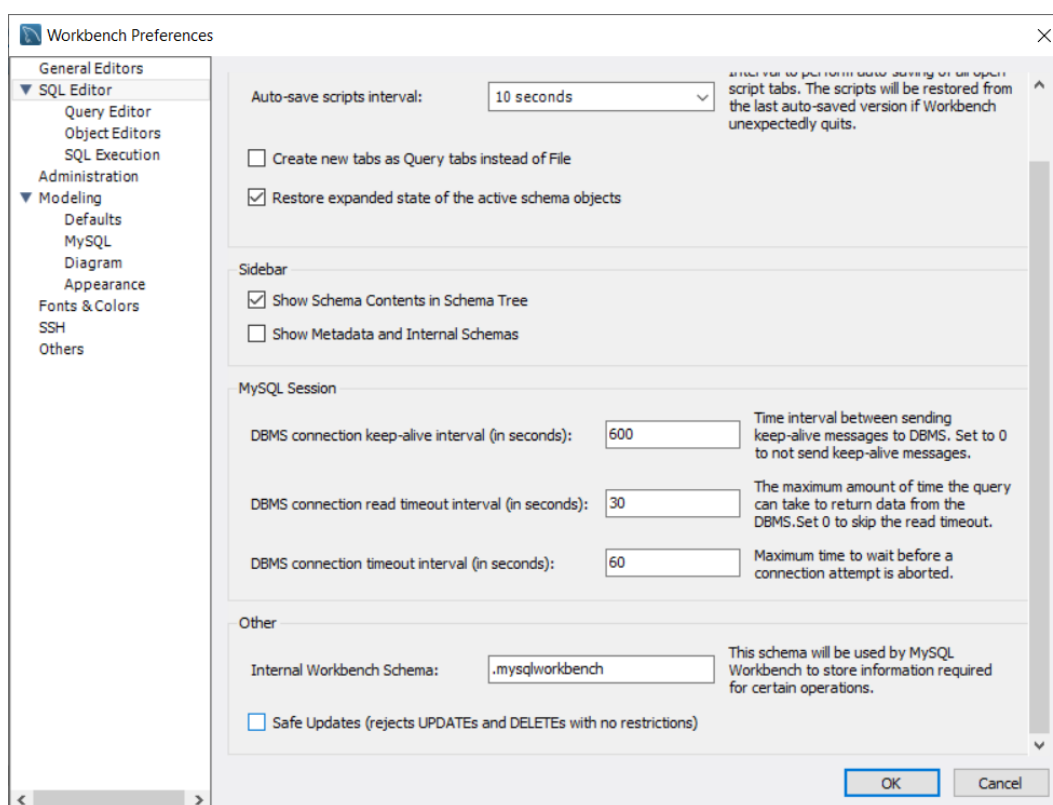
Depois de verificar se existem dados para excluir, apague-os a todos utilizando a seguinte frase:

```
DELETE FROM campeonato;
```

Ao fazê-lo a partir do MySQL Workbench, provavelmente aparecerá um erro a indicar que não foi possível executar a referida instrução, dado que está a ser utilizado o modo protegido. Isso significa que cada instrução DELETE deve ter uma cláusula WHERE:

```
Error Code: 1175. To disable safe mode, toggle the option in
Preferences -> SQL Editor -> Query Editor and reconnect.
```

```
You are using safe update mode and you tried to update a table without a
WHERE that uses a KEY column
```



Para desligar o modo protegido, basta ir ao menu “Edit” e clicar em “Preferências”, o que abrirá uma nova janela. No menu do lado esquerdo, em “SQL Editor”, deve desativar a opção “Safe updates (rejects UPDATES and DELETES with no restrictions)”, tal como mostra a imagem. Depois de clicar em “OK”, é ainda necessário reconectar-se. Pode fazê-lo ligando e desligando o Workbench ou clicando em “Query” no menu superior e depois em “Reconnect to Server”. Tente fazer de novo a seguinte consulta:

```
DELETE FROM campeonato;
```

Desta vez, todas as linhas da tabela já devem ter sido excluídas. Poderá confirmá-lo com a consulta seguinte, que, desta vez, deve devolver 0 resultados.

```
SELECT COUNT(*) FROM campeonato;
```

Como referido, há cada vez mais sistemas de segurança que impedem que se cometam descuidos. Sugerimos não desativar esta segurança, pois pode evitar muitos transtornos.

Se o que pretende fazer é apagar todos os registos de uma tabela, existe uma instrução específica para isso, muito mais eficiente do que a anterior – a instrução **TRUNCATE TABLE**. Esta é uma instrução mais eficiente, porque destrói e recria a tabela, o que é mais rápido do que excluir todos os registos da tabela. Podem ser encontradas mais informações sobre esta instrução em <http://dev.mysql.com/doc/refman/5.1/en/truncate-table.html>. A sua sintaxe é a seguinte:

```
TRUNCATE TABLE nomeTabela;
```

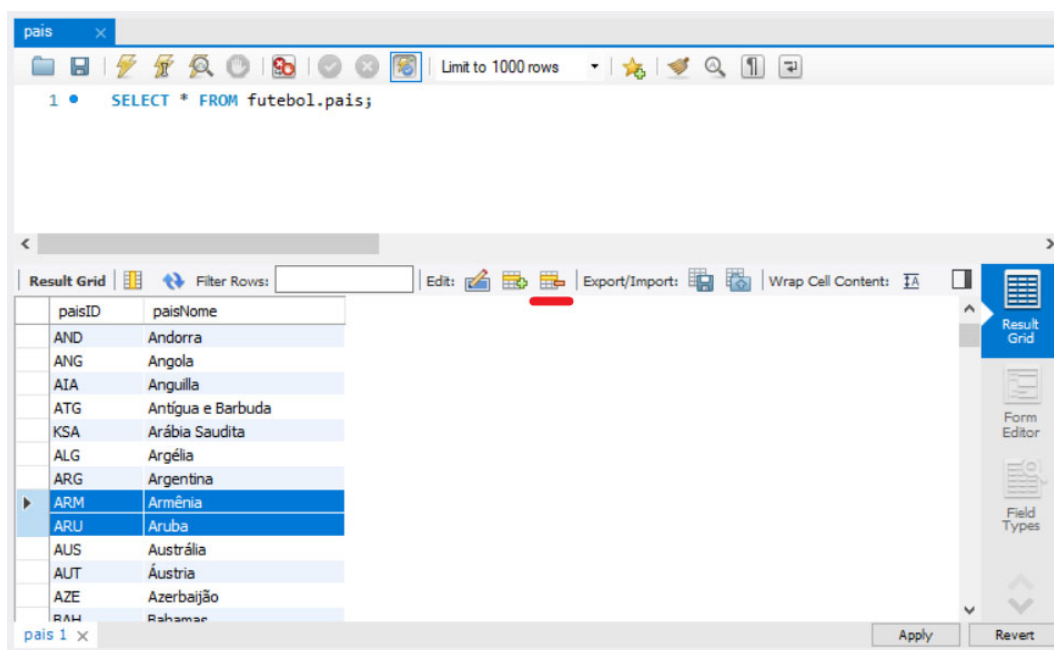
Tente limpar a tabela de registos “temporada” com esta nova instrução e verá como a tabela é limpa rapidamente.

```
TRUNCATE TABLE temporada;
```

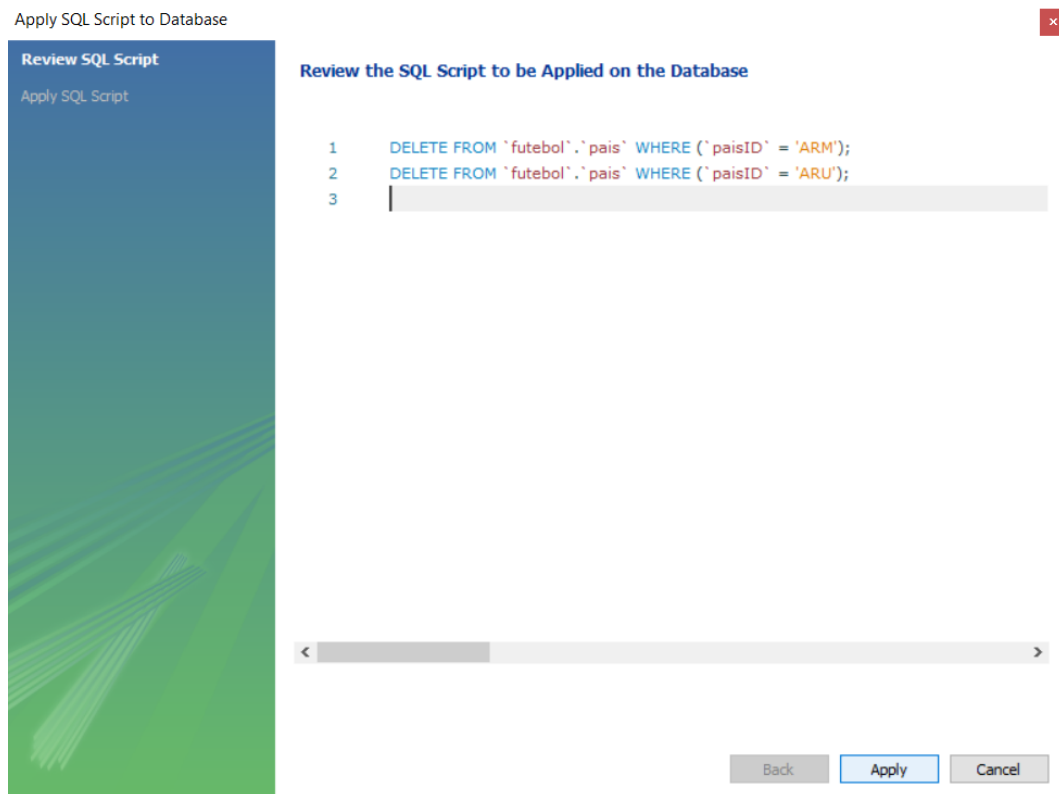
Se tentar fazer o mesmo com a tabela “pais”, observará que o programa impede de o fazer, indicando que está a quebrar uma das restrições de chave estrangeira impostas na tabela “equipa”. É exatamente o mesmo que aconteceu ao tentar eliminar a entrada “Itália” da base de dados.

```
TRUNCATE TABLE pais;  
Error Code: 1451  
Cannot delete or update a parent row: a foreignkey  
constraint fails ('futebol'. 'equipa', CONSTRAINT 'pais_equipa_FK'  
FOREIGN KEY ('paisID') REFERENCES 'pais' ('paisID') ON DELETE NO  
ACTION ON UPDATE CASCADE)
```

Até agora, aprendeu apenas a exclusão de registos por código, mas pode tirar proveito do MySQL Workbench e dos recursos gráficos que apresenta para excluir registos. Para isso, basta selecionar as linhas que devem ser excluídas e depois clicar no ícone com um símbolo de menos, designado “Delete selected lines”. Desta forma, o MySQL Workbench encarrega-se de realizar as consultas correspondentes, para apagar os dados de forma transparente para o utilizador.



Depois de clicar nesse ícone, as linhas desaparecem do ecrã, mas ainda não foram removidas da tabela. As alterações feitas só terão efeito quando clicar em "Apply". Nesse momento, aparecerá uma janela como a da imagem a seguir, com as consultas a realizar, e será solicitada a confirmação para que as alterações tenham efeito. Como sempre, a exclusão de linhas só será realizada se não for contra nenhuma das restrições de integridade.

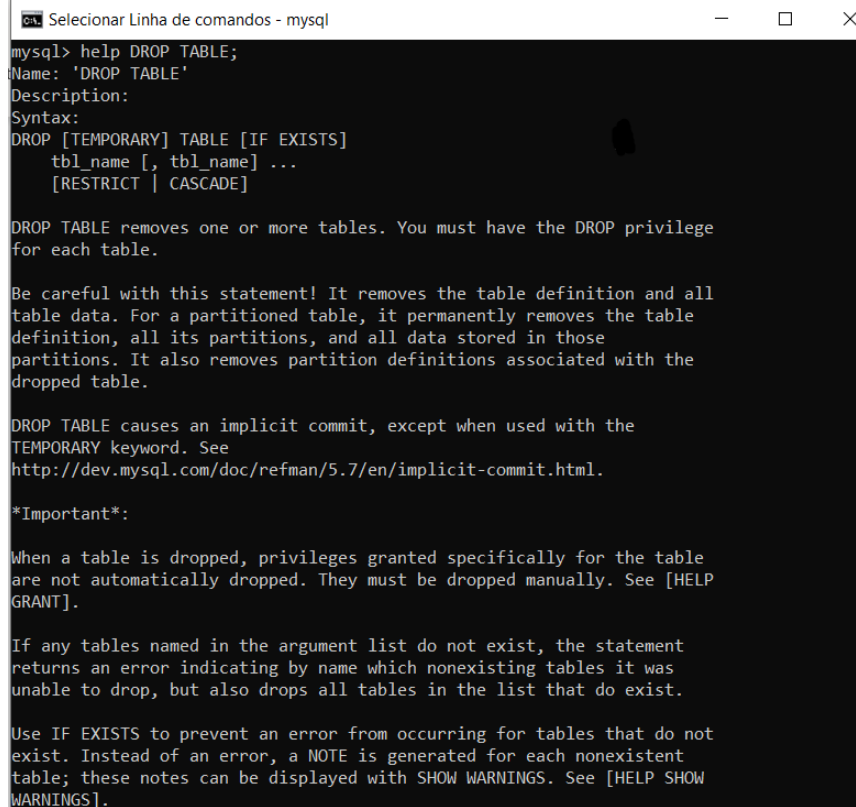


Além de eliminar linhas de uma tabela, pode eliminar as restrições de chave estrangeira, índices, etc., através das instruções **ALTER TABLE** combinadas com **DROP**.

Em resumo, as possibilidades de eliminar dados das tabelas de bases de dados são muitas, e os dados perdidos não são recuperáveis. Também aprendeu que as restrições de integridade fazem todo o sentido com tentativas de exclusão de dados. No próximo ponto, dará mais um passo e tentará eliminar não apenas dados, mas também as tabelas que os contêm.

3. APAGAR TABELAS

Por vezes, pode ser interessante excluir uma tabela que já não está a ser utilizada ou que não faz sentido no design da base de dados. Nestes casos, deverá usar a instrução **DROPTABLE**, sobre a qual poderá encontrar mais informações em <http://dev.mysql.com/doc/refman/5.1/en/drop-table.html>. A sua sintaxe é a seguinte:



```
mysql> help DROP TABLE;
Name: 'DROP TABLE'
Description:
Syntax:
DROP [TEMPORARY] TABLE [IF EXISTS]
      tbl_name [, tbl_name] ...
      [RESTRICT | CASCADE]

DROP TABLE removes one or more tables. You must have the DROP privilege
for each table.

Be careful with this statement! It removes the table definition and all
table data. For a partitioned table, it permanently removes the table
definition, all its partitions, and all data stored in those
partitions. It also removes partition definitions associated with the
dropped table.

DROP TABLE causes an implicit commit, except when used with the
TEMPORARY keyword. See
http://dev.mysql.com/doc/refman/5.7/en/implicit-commit.html.

*Important*:

When a table is dropped, privileges granted specifically for the table
are not automatically dropped. They must be dropped manually. See [HELP
GRANT].

If any tables named in the argument list do not exist, the statement
returns an error indicating by name which nonexistent tables it was
unable to drop, but also drops all tables in the list that do exist.

Use IF EXISTS to prevent an error from occurring for tables that do not
exist. Instead of an error, a NOTE is generated for each nonexistent
table; these notes can be displayed with SHOW WARNINGS. See [HELP SHOW
WARNINGS].
```

```
DROP [TEMPORARY] TABLE [IF EXISTS]
nomeTabela1 [, nomeTabela2] ...
[RESTRICT | CASCADE]
```

A cláusula opcional **IF EXISTS** permite que, se tentar apagar uma tabela que não existe, seja exibido um aviso e não um erro. No exemplo seguinte, tentará apagar-se uma tabela que nunca existiu da base de dados: “marcadores”.

```
DROP TABLE marcadores;
Error Code: 1051
Unknown table 'marcadores'
```

Se, por outro lado, adicionar IF EXISTS à consulta anterior, vai evitar este erro; será exibido um aviso a indicar que não houve alterações na base de dados.

Depois de tentar apagar a tabela que nunca existiu, tente eliminar uma tabela que existe, como a tabela “pais”. Para o fazer, execute a seguinte consulta de exclusão:

```
DROP TABLE pais;
```

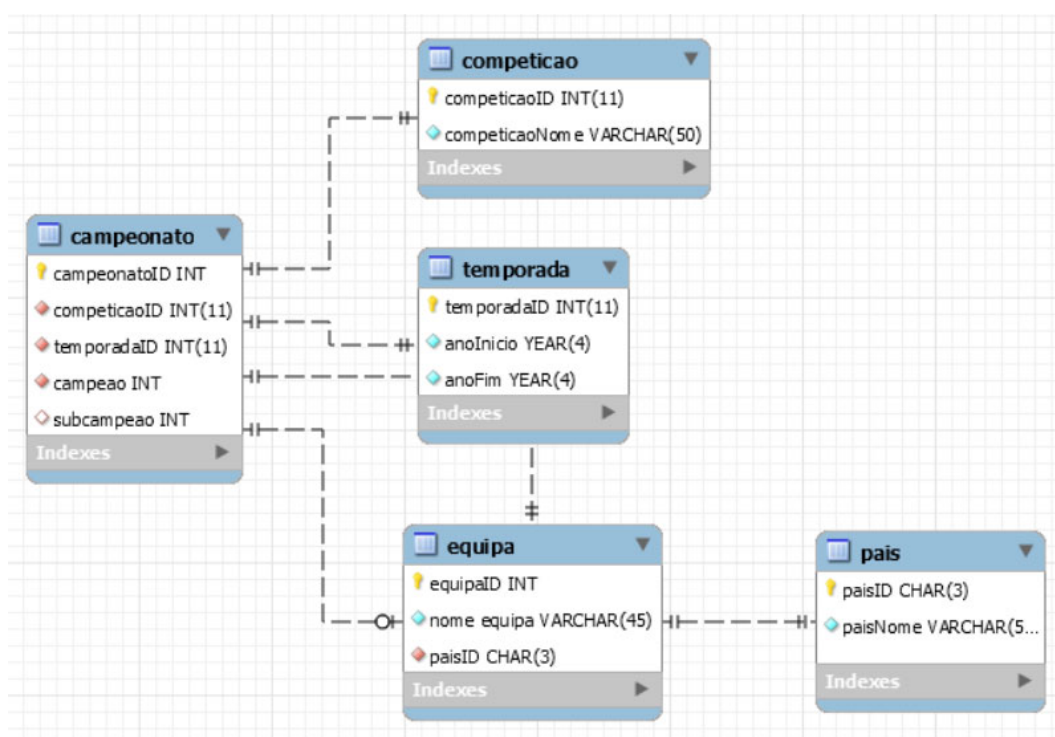
Algo falhará. Certamente, já sabe o que é: está impedido pelas restrições de chave estrangeira, ou seja, outras tabelas usam a tabela “pais”.

```
Error Code: 1217
Cannot delete or update a parent row: a foreignkey
constraint fails
```

A melhor maneira de saber como algumas tabelas estão relacionadas entre si e, portanto, como as eliminar, é a partir de um diagrama de relações entre as tabelas. Na imagem abaixo pode ver um diagrama com as relações entre as tabelas que pode ajudar a decidir a ordem correta de eliminação.

Como pode ver, a tabela “temporada” é utilizada pela tabela “campeonato”. A tabela “competicao” também é utilizada pela tabela “campeonato”. A tabela “pais” é usada pela tabela “equipa”. E a tabela “equipa” é usada pela tabela “campeonato”. Por último, a tabela “campeonato” não é utilizada por nenhuma outra tabela.

O termo “uso/utilização” não é o mais adequado, mas é bem compreendido; o termo “relacionamento” entre tabelas é muito mais apropriado. Estes relacionamentos são estabelecidos por meio de chaves estrangeiras. Portanto, se pretender eliminar uma tabela, nem sempre é necessário eliminar uma tabela primeiro e depois a outra; pode ser suficiente primeiro remover a restrição de chave estrangeira da tabela correspondente e, em seguida, remover a outra tabela.



Por exemplo, se pretender eliminar a tabela “temporada”, será suficiente remover a restrição de chave estrangeira que a relaciona à tabela “campeonato” antes de prosseguir com a eliminação.

```
ALTER TABLE campeonato  
DROP FOREIGN KEY temporada_campeonato_FK;  
DROP TABLE temporada;
```

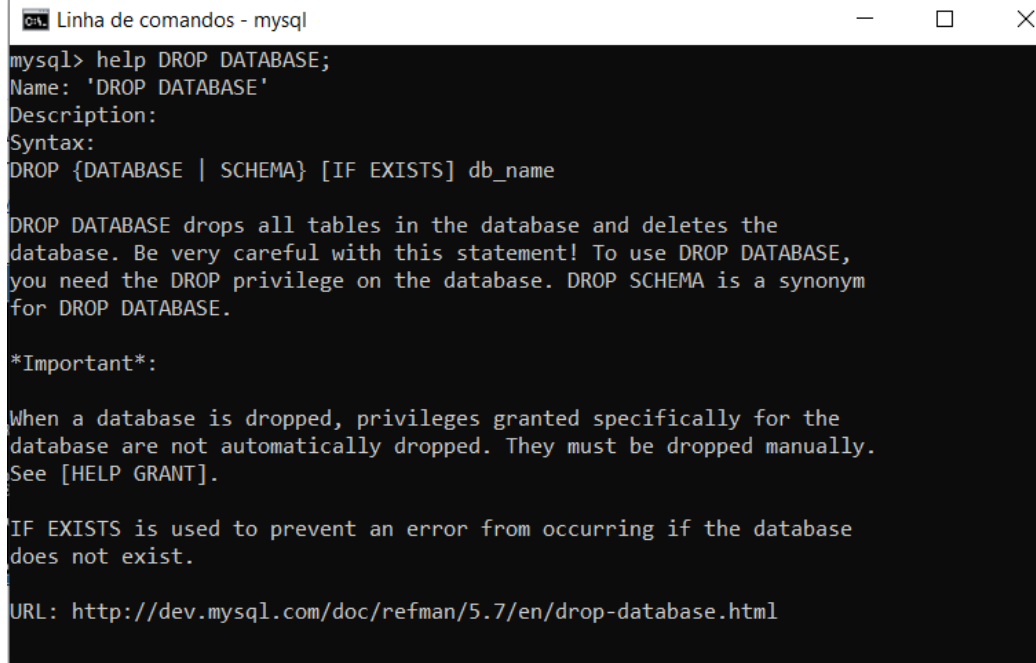
Como resultado da execução das instruções anteriores, a tabela “temporada” terá sido eliminada. Se pretender apagar as tabelas que faltam, deverá seguir as regras de chave estrangeira vistas neste exemplo ou apagar as tabelas na ordem correta, ou seja, na ordem inversa da sua criação: primeiro apagam-se as tabelas que não são chaves estrangeiras de nenhuma outra.

Neste caso, a ordem para poder eliminar todas as tabelas sem ter de eliminar as chaves estrangeiras é a seguinte: primeiro a tabela “campeonato”, depois as tabelas “competição”, “temporada” e “equipa” e, por último, a tabela “pais”.

```
DROP TABLE campeonato;  
DROP TABLE IF EXISTS temporada;  
DROP TABLE competicao;  
DROP TABLE equipa;  
DROP TABLE pais;
```

4. APAGAR BASES DE DADOS

Se o que pretende fazer é excluir uma base de dados, pode saltar todos os pontos anteriores e usar os comandos **DROP DATABASE** e **DROP SCHEMA**. Pode encontrar mais informações sobre estes comandos em <http://dev.mysql.com/doc/refman/5.1/en/drop-database.html>.



```
mysql> help DROP DATABASE;
Name: 'DROP DATABASE'
Description:
Syntax:
DROP {DATABASE | SCHEMA} [IF EXISTS] db_name

DROP DATABASE drops all tables in the database and deletes the
database. Be very careful with this statement! To use DROP DATABASE,
you need the DROP privilege on the database. DROP SCHEMA is a synonym
for DROP DATABASE.

*Important*:

When a database is dropped, privileges granted specifically for the
database are not automatically dropped. They must be dropped manually.
See [HELP GRANT].

IF EXISTS is used to prevent an error from occurring if the database
does not exist.

URL: http://dev.mysql.com/doc/refman/5.7/en/drop-database.html
```


Como pode ver na imagem acima, a sintaxe que permite excluir uma base de dados e todas as suas tabelas é uma das seguintes:

```
DROP DATABASE [IF EXISTS] nomeBaseDados;  
DROP SCHEMA [IF EXISTS] nomeBaseDados;
```

Tal como acontece com a instrução **DROP TABLE**, pode usar-se **IF EXISTS** para o mesmo propósito, embora, como foi referido antes, geralmente, não se tenta excluir algo que não existe.

Neste caso, se executar qualquer uma das instruções anteriores adaptada à base de dados “futebol” e, em seguida, atualizar a janela do MySQL Workbench, verificará que esta desaparece do servidor. Na segunda frase, incluiu-se IF EXISTS, para a eventualidade de se executar as duas frases em sequência:

```
DROP DATABASE futebol;  
DROP SCHEMA IF EXISTS futebol;
```

Por fim, poderá verificar, do lado esquerdo, que a base de dados “futebol” desapareceu. Foram destruídos todos os dados com os quais trabalhou durante estas unidades didáticas.

CONCLUSÃO

Ao longo desta unidade didática, foram gradualmente eliminadas as entidades da base de dados. Como viu, uma vez excluídas, não é possível voltar atrás. As sintaxes utilizadas para apagar dados são bastante simples, principalmente quando comparadas às sintaxes das frases abordadas nas unidades anteriores.

A dificuldade não está nas instruções em si, mas em pensar no que se pode ou não eliminar. As restrições de chave estrangeira são muito importantes ao tentar excluir registos e tabelas. Com esta unidade, completou o estudo sobre manipulação e modificação de dados.

AUTOAVALIAÇÃO

1. **Qual das seguintes afirmações permite excluir uma tabela?**
 - a) ERASE nomeTabela;
 - b) DELETE nomeTabela;
 - c) DROP nomeTabela;
 - d) DROP TABLE nomeTabela;

2. **Qual das seguintes afirmações sobre a instrução DELETE FROM nome-Tabela é verdadeira?**
 - a) Esta instrução não existe.
 - b) Esta instrução vai apagar a tabela indicada.
 - c) Esta instrução apagará todos os registos da tabela indicada, desde que esses registos possam ser excluídos e o modo de segurança não esteja ativo.
 - d) Esta instrução vai excluir a tabela e todos os seus registos.

3. **Qual das seguintes afirmações permite excluir uma base de dados?**
 - a) DROP SCHEMA nomeBaseDados.
 - b) ERASE nomeBaseDados.
 - c) DELETE nomeBaseDados.
 - d) DESTROY nomeBaseDados.

4. Qual das seguintes afirmações sobre a instrução **DROP DATABASE IF EXISTS nomeBaseDados;** é verdadeira?
- a) Esta instrução não existe.
 - b) Esta instrução exclui a base de dados, apenas se existir.
 - c) Esta instrução exclui sempre a base de dados.
 - d) Se a base de dados a ser excluída não existir, ocorre um erro.
5. Qual das seguintes afirmações não remove todos os registos da tabela?
- a) `DELETE FROM temporada WHERE '1' = '1';`
 - b) `DELETE FROM temporada WHERE anoInicio = 1955.`
 - c) `DELETE FROM temporada.`
 - d) `TRUNCATE TABLE temporada.`
6. Qual das seguintes afirmações é mais adequada para remover todos os registos da tabela “temporada”?
- a) `DELETE FROM temporada WHERE '1' = '1';`
 - b) `DELETE FROM temporada WHERE anoInicio = 1955.`
 - c) `DELETE FROM temporada.`
 - d) `TRUNCATE TABLE temporada.`
7. O que é que acontece se houver uma base de dados chamada “futebol” e executar as instruções **DROP DATABASE futebol;** e **DROP SCHEMA IF EXISTS futebol;**
- a) A base de dados “futebol” é excluída e, em seguida, aparece um erro indicando que a base de dados não pôde ser excluída porque não existia.
 - b) Exclui-se a base de dados “futebol” e, em seguida, um esquema que por acaso também tinha o mesmo nome.
 - c) Em primeiro lugar, elimina a base de dados e, em seguida, tenta eliminá-la novamente, mas como foi utilizado **IF EXISTS** na consulta, não aparecerá nenhum erro, apenas um aviso.
 - d) Nada, porque essas frases não existem.

8. Qual das seguintes palavras permite remover uma chave estrangeira de uma tabela com **ALTER TABLE**?
- a) DROP FOREIGN KEY.
 - b) DELETE FOREIGN KEY.
 - c) REMOVE FOREIGN KEY.
 - d) ERASE FOREIGN KEY.
9. Em que ordem é que as tabelas de uma base de dados devem ser excluídas se pretender excluir todas as tabelas com o menor número de problemas possível?
- a) Na mesma ordem em que foram criadas.
 - b) Na ordem inversa da criação.
 - c) Por ordem alfabética.
 - d) A ordem não importa.
10. O que é que acontece se o comando **DROP DATABASE** futebol; for executado?
- a) Registos, índices, tabelas e toda a base de dados são excluídos.
 - b) Registos, índices e tabelas são excluídos.
 - c) Os registos são excluídos.
 - d) As tabelas são eliminadas.

SOLUÇÕES

1.	d	2.	c	3.	a	4.	b	5.	b
6.	d	7.	c	8.	a	9.	b	10.	a

PROPOSTAS DE DESENVOLVIMENTO DO ESTUDO

Para expandir os conhecimentos adquiridos nesta unidade, recomenda-se que visite a seguinte página web:

- https://www.w3schools.com/sql/sql_delete.asp

BIBLIOGRAFIA

- Beaulieu, A. (2006), *Aprende SQL*. Madrid: Anaya Multimedia.
- Gutiérrez Gallardo, J. D. (2009), *MySQL 5.1*. Madrid: Anaya Multimedia.
- Oracle (2021), "Reference Manual". Disponível em: <http://dev.mysql.com/doc/refman/5.1/en/>.

Consultado a 27 de abril de 2021.

